Practical 1

Aim: Implement concurrent echo client-server application in java.

Theory: TCP stands for Transmission Control Protocol, a communications standard that enables application programs and computing devices to exchange messages over a network. It is designed to send packets across the internet and ensure the successful delivery of data and messages over networks. TCP organizes data so that it can be transmitted between a server and a client. It guarantees the integrity of the data being communicated over a network. Before it transmits data, TCP establishes a connection between a source and its destination, which it ensures remains alive until communication begins. It then breaks large amounts of data into smaller packets, while ensuring data integrity is in place throughout the process.

Code: We make two files.

Tcpserver.java:

```
import java.io.*;
import java.net.*;

public class TcpServer {
 public static void main(String[] args) throws Exception {
ServerSocket ss=new ServerSocket(8088); //Server is sitting at port
8088 and waiting for a client.
 System.out.println("server is ready!");
 Socket ls=ss.accept(); // accept() waits for a client.When client
connects, ls represents that connection.

// While part This means the server keeps repeating the communication
code.So after one message, it can again try to read another message.
while (true){
 System.out.println("Client Port is "+ls.getPort());
//ls.getPort() gives the port number of the connected client.

 //READING DATA FROM CLIENT
 InputStream is=ls.getInputStream(); InputStream = receive/read
Server wants to receive data from client. Therefore it uses:

 byte data[]=new byte[50]; //So server needs a byte array to store
those incoming bytes.
 is.read(data);
 String mfc=new String(data);//This converts the received byte array
into a String.
 //mfc: message from client
```

```
 mfc=mfc.trim(); This removes extra spaces from the beginning
and end of the received message.
```

Why is this needed?

Because earlier you created:

```
byte data[] = new byte[50];
```

So the buffer has space for 50 bytes, but the actual message may be much smaller.

Conceptually:

Before trim:

"Varun "

After trim:

"Varun"

```
 String mts="The message was:"+mfc; This creates the reply that the
server wants to send back.
 //mfs: message from server
 //SENDING MSG TO CLIENT
 OutputStream os=ls.getOutputStream();The server wants to send data
through the connected socket ls.
```

So it gets an **OutputStream**.

Remember:

InputStream = receive

OutputStream = send

```
 os.write(mfs.getBytes()); //Convert the server reply into bytes
and send it to the client.
 }
 }
}
```

Tcpclient.java:

```
import java.net.*;
```

```
import java.io.*;
class TcpClient {
 public static void main(String[] args) throws Exception {
System.out.println("connecting to server");
 Socket cs=new Socket("localhost",8088); //connects the client to the
server running on the same computer at port 8088

 BufferedReader br=new BufferedReader(new InputStreamReader(
System.in)); //Creates an input reader to take text from the
keyboard.
 System.out.println("The Local Port "+cs.getLocalPort()+"\nThe Remote
Port"+cs.getPort());
 System.out.println("The Local socket is "+cs);
System.out.println("Enter your name");
 String str=br.readLine();Reads the name entered by the user and
stores it in str.
 //SENDING DATA TO SERVER
 OutputStream os=cs.getOutputStream();Gets the output stream of the
client socket cs, which is used to send data from the client to the
server.
 os.write(str.getBytes()); //Converts the entered String into bytes
and sends it to the server.


 //READING DATA FROM SERVER
 InputStream is=cs.getInputStream(); Gets the input stream of the
client socket, which is used to receive data sent by the server.
byte data[]=new byte[50]; Creates a byte array of size 50 to
temporarily store the reply coming from the server.
 is.read(data);
 //PRINTING MESSAGE ON CLIENT CONSOLE
 String mfs=new String(data);
 mfs=mfs.trim();
 System.out.println(mfs);
 }
}
```

Output:

```
connecting to server
The Local Port 51722
The Remote Port8088
The Local socket is
```

```
Socket[addr=localhost/127.0.0.1,port=8088,localport=51722]
Enter your name
Varun
The message was: Varun
```

Explanation:

InputStream

OutputStream

Easy rule: **InputStream = data receive, OutputStream = data send.**

InputStream is = ls.getInputStream(); **data receive from client**

is.read(data); server is reading message from client Server

flow:

Wait

↓

Accept client

↓

Receive message

↓

Create reply

↓

Send reply

Client side

str

↓

getBytes()

↓

OutputStream

↓

SERVER

SERVER reply

↓

InputStream

↓

byte data[]

↓

String mfs

↓

trim()

↓

print